LAB 4: CÁC KIỂU DỮ LIỆU NÂNG CAO TRONG C
VÀ THUẬT TOÁN SẮP XẾP

I. Mục tiêu

- Hiểu và sử dụng được các kiểu dữ liệu nâng cao trong C (Advanced C types) như struct, enum, union, và typedef để tổ chức dữ liệu một cách hiệu quả.
- Trình bày và phân tích được cách bố trí dữ liệu trong bộ nhớ (memory layout), bao gồm vấn đề padding và ảnh hưởng đến hiệu năng.
- Sử dụng thành thạo con trỏ tới cấu trúc (pointer to struct), bao gồm truy cập dữ liệu thông qua toán tử ->.
- Hiểu khái niệm con trỏ hàm (function pointer) và áp dụng trong việc truyền hàm như một tham số.
- Sử dụng được hàm qsort trong thư viện chuẩn để sắp xếp dữ liệu với hàm so sánh tùy chỉnh.
- Cài đặt và phân tích các thuật toán sắp xếp cơ bản (bubble sort, selection sort, insertion sort).

II. Lý thuyết cơ bản

1. Kiểu dữ liệu nâng cao trong C

Trong các chương trình C cơ bản, dữ liệu thường được biểu diễn bằng các kiểu nguyên thủy như int, char, float hoặc các mảng. Tuy nhiên, khi bài toán trở nên phức tạp hơn, việc chỉ sử dụng các kiểu dữ liệu này sẽ không còn phù hợp. Nhiều đối tượng trong thực tế bao gồm nhiều thuộc tính khác nhau nhưng lại có liên hệ chặt chẽ với nhau, chẳng hạn như thông tin của một sinh viên, một điểm trong không gian, hay một bản ghi dữ liệu.

Để tổ chức dữ liệu một cách hợp lý, rõ ràng và dễ quản lý, ngôn ngữ C cung cấp một số kiểu dữ liệu nâng cao như struct, enum, typedef và union. Các công cụ này giúp lập trình viên mô hình hóa dữ liệu tốt hơn, giảm lỗi logic và tăng khả năng bảo trì của chương trình. Nội dung phần này tập trung trình bày các vấn đề khi dùng mảng rời rạc và cách khắc phục bằng các kiểu dữ liệu nâng cao.

1.1 Vấn đề của mảng: dữ liệu bị tách rời

Một cách tiếp cận phổ biến khi mới học lập trình là sử dụng nhiều mảng song song để lưu trữ các thuộc tính của một đối tượng. Ví dụ, để quản lý danh sách sinh viên, ta có thể dùng các mảng riêng biệt như:

```
int ids[100];
char names[100][50];
float gpas[100];
```

Trong cách tổ chức này, phần tử thứ i của mỗi mảng được xem là thông tin của cùng một sinh viên. Tuy nhiên, phương pháp này tồn tại nhiều hạn chế nghiêm trọng.

Trước hết, dữ liệu của một thực thể bị tách rời thành nhiều cấu trúc khác nhau trong bộ nhớ. Mối liên hệ giữa các trường dữ liệu chỉ được duy trì thông qua chỉ số mảng, chứ không được thể hiện trực tiếp trong cấu trúc dữ liệu. Điều này khiến chương trình trở nên khó hiểu và dễ sai sót.

Thứ hai, khi thực hiện các thao tác như hoán đổi, sắp xếp hoặc xóa phần tử, lập trình viên phải đảm bảo cập nhật đồng thời tất cả các mảng liên quan. Nếu chỉ thao tác trên một mảng mà quên các mảng khác, dữ liệu sẽ bị sai lệch nhưng chương trình vẫn chạy bình thường, gây ra lỗi logic rất khó phát hiện.

Thứ ba, việc truyền dữ liệu vào hàm trở nên phức tạp. Một thực thể phải được truyền dưới dạng nhiều tham số riêng lẻ thay vì một đối tượng duy nhất, làm giảm tính rõ ràng và khả năng tái sử dụng của mã nguồn.

Từ các vấn đề trên, có thể thấy rằng mảng không phù hợp để biểu diễn các thực thể có nhiều thuộc tính khác kiểu. Cần có một cơ chế cho phép gom nhóm các dữ liệu liên quan lại với nhau thành một đơn vị logic. Cơ chế đó chính là struct.

1.2 Struct (Cấu trúc)

struct (structure) là một kiểu dữ liệu do người lập trình định nghĩa, cho phép kết hợp nhiều trường dữ liệu (fields) có thể khác kiểu vào trong cùng một đối tượng. Nhờ đó, toàn bộ thông tin của một thực thể được lưu trữ một cách thống nhất và có tổ chức.

Cú pháp định nghĩa một cấu trúc như sau:

```
struct Student {
    char name[50];
    int age;
    float gpa;
    int id;
};
```

Sau khi định nghĩa, ta có thể khai báo biến thuộc kiểu này:

```
struct Student s1;
```

Hoặc vừa khai báo vừa khởi tạo:

```
struct Student s1 = {"Alice", 20, 3.8, 1001};
```

Các giá trị khởi tạo sẽ được gán theo thứ tự khai báo của các trường. Nếu không cung cấp đủ giá trị, các trường còn lại sẽ được gán về 0 tương ứng với kiểu dữ liệu của chúng.

Để truy cập các trường trong cấu trúc, ta sử dụng toán tử dấu chấm (.):

```
printf("%s %.2f\n", s1.name, s1.gpa);
s1.age = 21;
```

Toán tử này cho phép truy cập trực tiếp tới từng thành phần của đối tượng, giúp mã nguồn trở nên rõ ràng và dễ đọc.

Về mặt bộ nhớ, các trường của struct được lưu trữ liên tiếp theo thứ tự khai báo, tuy nhiên có thể tồn tại các byte đệm (padding) do yêu cầu căn chỉnh bộ nhớ (memory alignment). Mỗi kiểu dữ liệu thường phải bắt đầu tại một địa chỉ phù hợp với kích thước của nó (ví dụ int thường yêu cầu căn chỉnh theo 4 byte). Vì vậy, trình biên dịch có thể chèn thêm các byte trống giữa các trường hoặc ở cuối cấu trúc để đảm bảo hiệu năng truy cập của CPU.

Ví dụ:

```
struct B {
    char c;
    int n;
};
```

Mặc dù char chỉ chiếm 1 byte và int chiếm 4 byte, nhưng kích thước của struct B thường là 8 byte do có thêm 3 byte đệm để căn chỉnh trường int. Vì vậy, khi làm việc với cấu trúc, cần sử dụng toán tử sizeof để xác định kích thước thực tế thay vì tự tính toán.

Một struct có thể chứa các struct khác bên trong, được gọi là cấu trúc lồng nhau (nested struct). Ví dụ:

```
typedef struct {
    double x, y;
} Point;


typedef struct {
    Point center;
```

```
    double radius;
} Circle;
```

Khi đó, việc truy cập các trường sẽ thực hiện theo chuỗi:

c.center.x

Ngoài ra, ta có thể tạo mảng các cấu trúc để lưu nhiều đối tượng cùng loại:

struct Student roster[100];

Mỗi phần tử của mảng là một đối tượng hoàn chỉnh, giúp việc quản lý dữ liệu trở nên trực quan và an toàn hơn so với việc dùng nhiều mảng song song. Các phần tử trong mảng được lưu liên tiếp trong bộ nhớ, giúp việc duyệt và xử lý dữ liệu hiệu quả hơn.

1.3 Enumerations & typedef (Kiểu liệt kê và bí danh kiểu)

enum (enumeration) là kiểu dữ liệu cho phép định nghĩa một tập hợp các hằng số nguyên có tên. Kiểu này thường được sử dụng khi một biến chỉ có thể nhận một số giá trị xác định trước.

Ví dụ:

```
typedef enum {
    SUN, MON, TUE, WED, THU, FRI, SAT
} Day;
```

Theo mặc định, các giá trị sẽ được gán tự động bắt đầu từ 0 và tăng dần. Ta có thể sử dụng kiểu này như sau:

Day today = WED;

Việc sử dụng enum giúp chương trình dễ đọc hơn so với việc dùng các số nguyên trực tiếp. Thay vì sử dụng các giá trị như 3 hay 5, ta có thể dùng các tên có ý nghĩa như WED hoặc FRI, giúp người đọc hiểu rõ logic của chương trình.

So với #define, enum có nhiều ưu điểm hơn. #define chỉ là phép thay thế văn bản trước khi biên dịch và không có kiểm tra kiểu. Trong khi đó, enum gắn với một kiểu dữ liệu cụ thể, giúp trình biên dịch có thể cảnh báo khi sử dụng sai giá trị. Ngoài ra, các giá trị của enum cũng hiển thị rõ ràng hơn trong quá trình debug.

typedef là từ khóa dùng để tạo bí danh cho một kiểu dữ liệu. Nó giúp rút gọn cú pháp và làm cho mã nguồn dễ đọc hơn. Ví dụ:

typedef unsigned long long uint64;

Khi đó, ta có thể sử dụng uint64 thay cho kiểu dài dòng ban đầu.

typedef thường được sử dụng kết hợp với struct để loại bỏ từ khóa struct khi khai báo biến:

```
typedef struct {
    char name[50];
    int age;
} Student;

Student s1;
```

Ngoài ra, typedef còn rất hữu ích khi làm việc với các kiểu phức tạp như con trỏ hàm, giúp mã nguồn trở nên rõ ràng và dễ hiểu hơn.

1.4 Union (Liên hợp)

union là một kiểu dữ liệu cho phép nhiều trường khác nhau dùng chung một vùng nhớ. Điều này có nghĩa là tại một thời điểm, chỉ có một trường trong union có ý nghĩa.

Cú pháp định nghĩa:

```
union Data {
    int i;
    float f;
    char str[20];
};
```

Kích thước của union bằng kích thước của trường lớn nhất. Tất cả các trường đều bắt đầu tại cùng một địa chỉ trong bộ nhớ.

Ví dụ:

```
union Data d;
d.i = 10;
d.f = 3.14;  // ghi đè lên dữ liệu trước đó
```

Sau khi gán d.f, giá trị của d.i không còn ý nghĩa nữa. Vì vậy, khi sử dụng union, lập trình viên phải tự quản lý việc trường nào đang chứa dữ liệu hợp lệ.

Một kỹ thuật phổ biến để giải quyết vấn đề này là sử dụng **tagged union** (union có nhãn). Ta kết hợp union với một biến enum để lưu thông tin về kiểu dữ liệu hiện tại:

```
typedef enum { INT_TYPE, FLOAT_TYPE } DataType;

typedef struct {
   DataType type;
   union {
      int i;
      float f;
   } data;
} TaggedData;
```

Khi đó, trường type cho biết dữ liệu trong union đang ở dạng nào. Trước khi truy cập, ta kiểm tra giá trị của type để đảm bảo đọc đúng trường. Cách tổ chức này giúp tăng tính an toàn và tránh lỗi khi làm việc với union.

union thường được sử dụng trong các hệ thống yêu cầu tiết kiệm bộ nhớ hoặc khi cần biểu diễn nhiều kiểu dữ liệu khác nhau trên cùng một vùng nhớ, chẳng hạn như trong lập trình hệ thống, giao thức mạng hoặc trình biên dịch.

2. Con trỏ tới cấu trúc

Trong phần trước, chúng ta đã biết struct là công cụ dùng để gom nhiều trường dữ liệu liên quan vào cùng một đơn vị. Tuy nhiên, trong thực tế lập trình, đặc biệt là khi làm việc với hàm, danh sách liên kết, cấp phát động hoặc các cấu trúc dữ liệu lớn, ta không chỉ thao tác với biến struct trực tiếp mà còn thường xuyên làm việc với **con trỏ tới cấu trúc** (pointer to struct).

Xét ví dụ đơn giản sau:

```
typedef struct {
   int x;
   int y;
} Point;
```

Ta có thể khai báo một biến cấu trúc và một con trỏ trỏ tới nó như sau:

```
Point p = {3, 7};
Point *pp = &p;
```

Ở đây, p là một biến kiểu Point, còn pp là một con trỏ chứa địa chỉ của p. Điều này có nghĩa là pp không lưu trực tiếp hai giá trị x và y, mà lưu địa chỉ bộ nhớ nơi biến p đang tồn tại. Muốn truy cập dữ liệu bên trong cấu trúc thông qua con trỏ, ta phải thực hiện truy cập gián tiếp.

Khi làm việc với biến struct trực tiếp, ta dùng toán tử dấu chấm (.):

```
p.x = 10;
p.y = 20;
```

Nhưng khi làm việc với con trỏ tới struct, ta dùng toán tử mũi tên (->):

```
pp->x = 20;
pp->y = 30;
```

Toán tử -> thực chất là cách viết rút gọn của biểu thức:

```
(*pp).x
(*pp).y
```

Trong đó, *pp là phép giải tham chiếu (dereference), lấy ra đối tượng Point mà pp đang trỏ tới, sau đó .x hoặc .y truy cập trường của đối tượng đó. Tuy nhiên, vì cách viết (*pp).x khá dài và dễ gây nhầm do phải dùng ngoặc, C cung cấp toán tử -> để biểu diễn gọn hơn. Đây là toán tử rất quan trọng và xuất hiện thường xuyên trong hầu hết các chương trình C có dùng cấu trúc dữ liệu động.

Cần phân biệt giữa toán tử dấu chấm và toán tử mũi tên như sau:

- Dùng . khi bên trái là biến struct trực tiếp
- Dùng -> khi bên trái là con trỏ tới struct

Nếu dùng sai, trình biên dịch sẽ báo lỗi vì kiểu dữ liệu không khớp.

Một câu hỏi quan trọng là: tại sao phải dùng con trỏ tới struct thay vì truyền nguyên cả struct vào hàm? Để trả lời, ta xét hai cách viết sau:

```
typedef struct {
   int x;
   int y;
} Point;

void scaleBAD(Point p, int factor) {
   p.x *= factor;
   p.y *= factor;
}
```

Trong hàm scaleBAD, tham số p được truyền theo giá trị (pass by value). Điều này có nghĩa là khi gọi hàm, toàn bộ dữ liệu của biến gốc sẽ được sao chép vào một biến cục bộ

mới tên là p. Mọi thay đổi trong hàm chỉ tác động đến bản sao này. Sau khi hàm kết thúc, bản sao bị hủy và dữ liệu gốc không thay đổi.

Ví dụ:

```
Point p = {3, 7};
scaleBAD(p, 2);
```

Sau lời gọi trên, biến p bên ngoài hàm vẫn giữ nguyên giá trị ban đầu. Đây là hệ quả trực tiếp của cơ chế truyền giá trị trong C.

Ngược lại, nếu muốn thay đổi trực tiếp dữ liệu gốc, ta phải truyền địa chỉ của biến vào hàm:

```
void scale(Point *p, int factor) {
   p->x *= factor;
   p->y *= factor;
}
```

Khi đó:

```
Point p = {3, 7};
scale(&p, 2);
```

Lời gọi &p truyền địa chỉ của biến p vào hàm. Bên trong hàm, con trỏ p trỏ tới đúng vùng nhớ của biến gốc, nên các phép gán p->x và p->y sẽ tác động trực tiếp lên dữ liệu bên ngoài. Sau khi hàm kết thúc, biến p thực sự đã được thay đổi.

Ngoài vấn đề thay đổi dữ liệu gốc, việc dùng con trỏ còn mang lại lợi ích về hiệu năng. Xét ví dụ:

```
typedef struct {
   double x, y, z;
} Vec3;
```

Một biến Vec3 chứa ba số thực double, thường chiếm 24 byte. Nếu truyền biến này theo giá trị, mỗi lần gọi hàm chương trình phải sao chép toàn bộ 24 byte dữ liệu. Nếu truyền con trỏ Vec3 *, hệ thống chỉ cần sao chép một địa chỉ, thường là 8 byte trên hệ 64 bit. Với các cấu trúc lớn hơn, chênh lệch này càng đáng kể.

Vì vậy, trong thực hành, có một quy tắc rất hữu ích: với các struct nhỏ, có thể truyền theo giá trị nếu chỉ cần đọc dữ liệu; nhưng với các struct lớn hoặc khi cần sửa dữ liệu gốc, nên truyền bằng con trỏ. Cách làm này vừa giảm chi phí sao chép vừa cho phép hàm thao tác trực tiếp trên đối tượng ban đầu.

Khi hàm chỉ cần đọc dữ liệu chứ không được phép sửa, ta thường dùng con trỏ hằng (const pointer to struct theo nghĩa dữ liệu hằng):

```
void printPoint(const Point *p) {
   printf("(%d, %d)\n", p->x, p->y);
}
```

Từ khóa const ở đây cho biết rằng hàm sẽ không thay đổi nội dung của đối tượng mà p trỏ tới. Điều này có hai ý nghĩa quan trọng. Thứ nhất, nó giúp trình biên dịch phát hiện lỗi nếu trong hàm có lệnh sửa dữ liệu. Thứ hai, nó giúp người đọc chương trình hiểu ngay rằng hàm này chỉ dùng để xem dữ liệu, không dùng để cập nhật dữ liệu.

Từ góc độ lập trình có cấu trúc, đây là một thói quen tốt nên duy trì: nếu một hàm không cần sửa dữ liệu thì nên khai báo tham số là const T *. Cách viết này vừa hiệu quả, vừa an toàn, vừa tăng tính rõ nghĩa của mã nguồn.

Một ứng dụng rất quan trọng khác của con trỏ tới struct là khi làm việc với bộ nhớ động. Ví dụ:

```
Point *p = malloc(sizeof(Point));
if (p != NULL) {
   p->x = 10;
   p->y = 20;
}
```

Ở đây, malloc cấp phát một vùng nhớ trên heap đủ để chứa một đối tượng kiểu Point, và trả về địa chỉ của vùng nhớ đó. Vì p là con trỏ, nên việc truy cập các trường phải dùng toán tử ->. Đây chính là nền tảng để xây dựng các cấu trúc dữ liệu như danh sách liên kết: mỗi nút thường được cấp phát động và được thao tác thông qua con trỏ.

Như vậy, có thể thấy rằng con trỏ tới struct là công cụ không thể thiếu khi lập trình C ở mức thực hành. Nó cho phép truyền cấu trúc vào hàm một cách hiệu quả, sửa trực tiếp dữ liệu gốc, thao tác với dữ liệu cấp phát động và là cơ sở để xây dựng các cấu trúc dữ liệu phức tạp

3. Function pointer (Con trỏ hàm)

Trong các phần trước, chúng ta đã quen với con trỏ dữ liệu, tức là các biến lưu địa chỉ của vùng nhớ chứa giá trị như int, double hay struct. Tuy nhiên, trong C, không chỉ dữ liệu mới có địa chỉ; hàm cũng có địa chỉ trong bộ nhớ. Điều này dẫn đến một khái niệm rất mạnh của ngôn ngữ C, đó là con trỏ hàm (function pointer).

Con trỏ hàm là một biến dùng để lưu địa chỉ của một hàm. Nhờ đó, chương trình có thể chọn hàm cần gọi tại thời điểm chạy thay vì cố định ngay từ đầu. Đây là nền tảng cho nhiều kỹ thuật quan trọng như callback, strategy pattern, xử lý sự kiện, bảng dispatch, plugin và đặc biệt là các hàm thư viện tổng quát như qsort.

Để hiểu rõ hơn, xét hai hàm đơn giản sau:

```
int add(int a, int b) {
   return a + b;
}

int mul(int a, int b) {
   return a * b;
}
```

Cả hai hàm này đều nhận hai đối số kiểu int và trả về một giá trị int. Vì có cùng kiểu chữ ký hàm (function signature), ta có thể khai báo một con trỏ hàm phù hợp để trỏ đến một trong hai hàm đó:

```
int (*op)(int, int);
```

Cú pháp này thường gây khó đọc với người mới học, vì nó kết hợp nhiều thành phần cùng lúc. Ta phân tích như sau:

- op là tên biến
- *op cho biết đây là con trỏ
- (int, int) cho biết con trỏ này trỏ đến một hàm nhận hai đối số kiểu int
- int ở đầu cho biết hàm được trỏ tới trả về int

Sau khi khai báo, ta có thể gán địa chỉ hàm cho con trỏ:

```
op = add;
printf("%d\n", op(3, 4));   // 7

op = mul;
printf("%d\n", op(3, 4));   // 12
```

Khi viết op = add;, ta đang gán địa chỉ của hàm add cho biến op. Sau đó, lời gọi op(3, 4) sẽ gián tiếp gọi đúng hàm mà op đang trỏ tới. Đây chính là đặc điểm quan trọng nhất của con trỏ hàm: cho phép trì hoãn quyết định chọn hàm đến thời điểm chạy.

Về mặt tư duy lập trình, điều này giúp tách rời thuật toán chung khỏi hành vi cụ thể. Thay vì viết nhiều phiên bản hàm gần giống nhau, ta có thể viết một hàm tổng quát và truyền vào hành vi cần thiết dưới dạng con trỏ hàm.

Ví dụ:

```
typedef int (*BinOp)(int, int);


int apply(int a, int b, BinOp fn) {
   return fn(a, b);
}
```

Khi đó:

```
printf("%d\n", apply(5, 3, add)); // 8
printf("%d\n", apply(5, 3, mul)); // 15
```

Ở đây, apply không quan tâm chi tiết phép toán là cộng hay nhân. Nó chỉ biết rằng tham số fn là một hàm nhận hai số nguyên và trả về một số nguyên. Chính hàm được truyền vào sẽ quyết định hành vi cụ thể. Đây là một ví dụ điển hình cho ý tưởng truyền chiến lược xử lý vào hàm.

Do cú pháp con trỏ hàm khá rối, trong thực hành người ta thường dùng typedef để đặt tên cho kiểu con trỏ hàm. Ví dụ:

```
typedef int (*BinOp)(int, int);
```

Sau đó, thay vì phải viết lại cú pháp phức tạp, ta chỉ cần dùng tên ngắn gọn BinOp. Đây là lý do vì sao typedef rất hữu ích khi đi cùng con trỏ hàm: nó giúp mã nguồn dễ đọc và ít gây sợ hãi hơn cho người mới học.

Một ứng dụng rất quan trọng của con trỏ hàm là callback. Callback có thể hiểu là một hàm được truyền vào hàm khác để được gọi lại khi cần. Ví dụ, một hàm xử lý danh sách có thể nhận vào một hàm khác để quyết định cách in từng phần tử; một thư viện giao diện có thể nhận vào hàm xử lý khi người dùng nhấn nút; một hàm sắp xếp tổng quát có thể nhận vào hàm so sánh do người dùng tự định nghĩa. Trong tất cả các trường hợp này, callback chính là một con trỏ hàm.

Điểm mạnh của callback là nó làm cho chương trình linh hoạt hơn. Thay vì gắn cứng một hành vi, ta có thể thay đổi hành vi bằng cách thay hàm được truyền vào. Đây là nguyên lý được dùng rất rộng rãi trong lập trình hệ thống và thư viện chuẩn C.

Tuy nhiên, con trỏ hàm cũng có một số điểm cần cẩn thận. Trước hết, chỉ những hàm có chữ ký phù hợp mới có thể gán cho cùng một con trỏ hàm. Nếu kiểu tham số hoặc kiểu trả về không khớp, trình biên dịch sẽ báo lỗi hoặc cảnh báo. Điều này rất quan trọng vì lời gọi gián tiếp qua con trỏ hàm đòi hỏi chương trình phải biết chính xác cách truyền đối số và cách nhận kết quả.

Thứ hai, một con trỏ hàm nếu chưa được khởi tạo thì không được gọi. Ví dụ:

```
BinOp fn;
printf("%d\n", fn(1, 2));   // lỗi nghiêm trọng nếu fn chưa được gán
```

Nếu gọi một con trỏ hàm chưa có giá trị hợp lệ, chương trình có thể bị lỗi chạy ngay lập tức. Vì vậy, trong các chương trình cẩn thận, người ta thường kiểm tra:

```
if (fn != NULL) {
   printf("%d\n", fn(1, 2));
}
```

Dù trong thực hành ngắn có thể ít gặp, đây vẫn là nguyên tắc quan trọng để tránh lỗi khó dò.

Một điểm cần nhấn mạnh là con trỏ hàm không giống con trỏ dữ liệu thông thường. Ta không nên làm số học trên con trỏ hàm, không nên giả định địa chỉ hàm có thể được thao tác như địa chỉ mảng. Mục tiêu chính của nó là lưu địa chỉ và gọi gián tiếp hàm phù hợp.

Như vậy, con trỏ hàm là công cụ giúp chương trình C trở nên linh hoạt hơn, cho phép truyền hành vi xử lý vào hàm khác, xây dựng callback và viết các hàm tổng quát.

4. qsort

Trong thực hành lập trình, sắp xếp (sorting) là một thao tác xuất hiện rất thường xuyên. Ta có thể cần sắp xếp mảng số nguyên, mảng số thực, danh sách sinh viên theo GPA, theo tên hoặc theo mã số. Một cách tiếp cận là tự cài đặt các thuật toán như bubble sort, selection sort hoặc insertion sort. Tuy nhiên, khi làm việc trong chương trình thực tế, C đã cung cấp sẵn một hàm sắp xếp tổng quát rất mạnh trong thư viện chuẩn, đó là qsort.

Hàm qsort nằm trong thư viện <stdlib.h>. Đây là hàm sắp xếp tổng quát, có thể áp dụng cho mọi loại mảng, miễn là người lập trình cung cấp một hàm so sánh phù hợp. Chính vì vậy, qsort là ví dụ kinh điển cho việc kết hợp giữa con trỏ void * và con trỏ hàm.

Khai báo tổng quát của qsort là:

```
void qsort(void *base, size_t n, size_t size,
           int (*cmp)(const void *, const void *));
```

Để hiểu đúng cách dùng qsort, cần phân tích từng tham số:

- base là con trỏ tới phần tử đầu tiên của mảng cần sắp xếp
- n là số phần tử của mảng
- size là kích thước của mỗi phần tử, thường viết bằng sizeof(...)
- cmp là hàm so sánh hai phần tử

Ý tưởng của qsort là: vì hàm này không biết kiểu dữ liệu cụ thể của mảng, nên nó làm việc trên mức byte. Còn việc xác định phần tử nào nhỏ hơn, bằng nhau hay lớn hơn sẽ do người dùng tự định nghĩa thông qua hàm so sánh.

Xét ví dụ sắp xếp mảng số nguyên tăng dần:

```
#include <stdio.h>
#include <stdlib.h>

int cmpInt(const void *a, const void *b) {
    int x = *(const int *)a;
    int y = *(const int *)b;

    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}

int main() {
    int nums[5] = {5, 2, 8, 1, 9};

    qsort(nums, 5, sizeof(int), cmpInt);

    for (int i = 0; i < 5; i++) {
        printf("%d ", nums[i]);
    }
    return 0;
}
```

Trong hàm cmpInt, hai tham số a và b có kiểu const void *. Điều này có nghĩa là chúng là các con trỏ chung, chưa có kiểu cụ thể. Vì ta biết mảng đang sắp xếp là mảng int, nên bên trong comparator ta phải ép kiểu chúng về const int *, sau đó giải tham chiếu để lấy giá trị thật.

Giá trị trả về của comparator phải tuân theo đúng hợp đồng sau:

- trả về số âm nếu phần tử thứ nhất nhỏ hơn phần tử thứ hai
- trả về 0 nếu hai phần tử bằng nhau
- trả về số dương nếu phần tử thứ nhất lớn hơn phần tử thứ hai

Đây là quy tắc rất quan trọng. Nếu comparator không tuân thủ quy ước này, qsort có thể cho ra kết quả sai hoặc không ổn định.

Một lỗi rất dễ mắc phải là viết comparator cho số nguyên theo kiểu:

return *(int *)a - *(int *)b;

Cách viết này ngắn, nhưng không an toàn. Nếu hai số nguyên có giá trị lớn, phép trừ có thể bị tràn số (overflow).

Không chỉ sắp xếp số nguyên, qsort còn rất hữu ích khi làm việc với mảng cấu trúc. Ví dụ:

```
typedef struct {
    char name[50];
    float gpa;
    int id;
} Student;
```

Nếu muốn sắp xếp mảng sinh viên theo GPA tăng dần, ta có thể viết:

```
int cmpGPA(const void *a, const void *b) {
    const Student *sa = (const Student *)a;
    const Student *sb = (const Student *)b;

    if (sa->gpa < sb->gpa) return -1;
    if (sa->gpa > sb->gpa) return 1;
    return 0;
}
```

và gọi:

qsort(roster, n, sizeof(Student), cmpGPA);

Điểm quan trọng ở đây là qsort không biết gì về Student. Nó chỉ biết rằng mảng có n phần tử, mỗi phần tử có kích thước sizeof(Student) byte. Mọi logic so sánh theo trường nào đều nằm trong comparator mà ta cung cấp.

Tương tự, nếu muốn sắp xếp theo tên alphabet, ta có thể dùng strcmp:

```
#include <string.h>

int byName(const void *a, const void *b) {
   const Student *sa = (const Student *)a;
   const Student *sb = (const Student *)b;
   return strcmp(sa->name, sb->name);
}
```

Khi đó, cùng một mảng roster, chỉ cần đổi comparator là ta có thể thay đổi tiêu chí sắp xếp. Đây là ưu điểm rất lớn của qsort: thuật toán sắp xếp là tổng quát, còn tiêu chí sắp xếp được tách riêng thành hàm so sánh.

Một điểm cần đặc biệt lưu ý khi so sánh số thực là không nên trừ trực tiếp rồi ép sang int. Ví dụ:

return ((Student *)a)->gpa - ((Student *)b)->gpa;

Cách viết này sai vì hiệu của hai số thực có thể rất nhỏ, khi ép sang int sẽ thành 0, làm comparator hoạt động sai. Ngoài ra, số thực còn có thể liên quan đến các trường hợp đặc biệt như NaN hoặc -0.0. Vì vậy, với float và double, luôn nên dùng phép so sánh > và < tường minh như ví dụ ở trên.

Cũng cần lưu ý rằng qsort chỉ quan tâm đến thứ tự do comparator quyết định, chứ không quan tâm nội dung cụ thể của phần tử. Khi sắp xếp mảng Student, toàn bộ từng cấu trúc Student sẽ được di chuyển trong bộ nhớ. Nói cách khác, qsort không chỉ đổi chỗ một trường nào đó, mà đổi chỗ cả bản ghi sinh viên. Điều này chính là ưu điểm của việc dùng struct: mỗi phần tử của mảng là một đơn vị dữ liệu hoàn chỉnh, nên việc sắp xếp sẽ giữ được tính nhất quán giữa các trường.

Về độ phức tạp, qsort thường được kỳ vọng hoạt động gần mức O(n log n)) trên trung bình, tốt hơn nhiều so với các thuật toán $O(n^2)$ như bubble sort hay selection sort khi dữ liệu lớn. Dù tên hàm là qsort, triển khai thực tế của thư viện chuẩn trên từng hệ thống có

thể khác nhau; người lập trình không nên phụ thuộc vào chi tiết cài đặt mà chỉ cần nắm cách sử dụng đúng.

5. Các thuật toán sắp xếp (Sorting Algorithms)

Sắp xếp (sorting) là một trong những bài toán cơ bản và quan trọng nhất trong khoa học máy tính. Nhiều bài toán thực tế yêu cầu dữ liệu phải được sắp xếp trước khi xử lý, ví dụ như tìm kiếm nhanh, thống kê, hoặc hiển thị dữ liệu theo thứ tự.

Trong phần này, chúng ta sẽ tìm hiểu ba thuật toán sắp xếp cơ bản:

- Bubble sort (sắp xếp nổi bọt)
- Selection sort (sắp xếp chọn)
- Insertion sort (sắp xếp chèn)

Các thuật toán này có ưu điểm là đơn giản, dễ cài đặt và phù hợp để hiểu bản chất của quá trình sắp xếp. Tuy nhiên, chúng có độ phức tạp cao và không phù hợp với dữ liệu lớn.

5.1 Bubble sort (Sắp xếp nổi bọt)

Bubble sort là thuật toán sắp xếp đơn giản nhất. Ý tưởng của thuật toán là lặp lại nhiều lần việc so sánh các cặp phần tử liền kề và hoán đổi chúng nếu chúng sai thứ tự. Sau mỗi vòng lặp, phần tử lớn nhất sẽ nổi lên cuối mảng, giống như bọt khí nổi lên trong nước, do đó có tên gọi là bubble sort.

Giả sử ta có mảng:

int a[] = {5, 2, 8, 1, 9};

Trong vòng lặp đầu tiên:

- So sánh 5 và 2 → đổi chỗ → {2, 5, 8, 1, 9}
- So sánh 5 và 8 → không đổi
- So sánh 8 và 1 → đổi chỗ → {2, 5, 1, 8, 9}
- So sánh 8 và 9 → không đổi

Sau vòng này, phần tử lớn nhất (9) đã ở đúng vị trí cuối.

Thuật toán tiếp tục lặp lại quá trình trên cho các phần tử còn lại.

Cài đặt:

```
void bubbleSort(int a[], int n) {
   for (int i = 0; i < n - 1; i++) {
      for (int j = 0; j < n - 1 - i; j++) {
         if (a[j] > a[j + 1]) {
            int tmp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = tmp;
         }
      }
   }
}
```

Trong đó:

- Vòng lặp ngoài (i) xác định số lần lặp
- Vòng lặp trong (j) thực hiện so sánh và hoán đổi

Một cải tiến đơn giản của bubble sort là thêm biến cờ (flag) để dừng sớm nếu mảng đã được sắp xếp:

```
void bubbleSort(int a[], int n) {
   for (int i = 0; i < n - 1; i++) {
      int swapped = 0;
      for (int j = 0; j < n - 1 - i; j++) {
         if (a[j] > a[j + 1]) {
            int tmp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = tmp;
            swapped = 1;
         }
      }
      if (!swapped) break;
   }
}
```

Độ phức tạp (time complexity):

- Trường hợp xấu nhất: $O(n^2)$
- Trường hợp tốt nhất (đã có thứ tự, có tối ưu): $O(n)$

Bubble sort là thuật toán ổn định (stable), nghĩa là các phần tử bằng nhau sẽ giữ nguyên thứ tự ban đầu.

5.2 Selection sort (Sắp xếp chọn)

Selection sort hoạt động dựa trên ý tưởng: tại mỗi bước, chọn phần tử nhỏ nhất trong phần chưa sắp xếp và đưa nó về đúng vị trí.

Thuật toán chia mảng thành hai phần:

- Phần đã sắp xếp (bên trái)
- Phần chưa sắp xếp (bên phải)

Ban đầu, phần đã sắp xếp rỗng. Ở mỗi bước, ta tìm phần tử nhỏ nhất trong phần chưa sắp xếp và đổi chỗ với phần tử đầu tiên của phần đó.

Ví dụ:

int a[] = {5, 2, 8, 1, 9};

Bước 1:

- Tìm min trong toàn mảng → 1
- Đổi chỗ với phần tử đầu → {1, 2, 8, 5, 9}

Bước 2:

- Tìm min trong {2, 8, 5, 9} → 2 → không đổi

Bước 3:

- Tìm min trong {8, 5, 9} → 5 → đổi chỗ

Cài đặt:

```
void selectionSort(int a[], int n) {
   for (int i = 0; i < n - 1; i++) {
      int minIndex = i;

      for (int j = i + 1; j < n; j++) {
         if (a[j] < a[minIndex]) {
            minIndex = j;
         }
      }
```

```
        if (minIndex != i) {
            int tmp = a[i];
            a[i] = a[minIndex];
            a[minIndex] = tmp;
        }
    }
}
```

Đặc điểm:

- Luôn thực hiện $n-1$lần chọn
- Mỗi lần chọn phải duyệt phần còn lại của mảng

Độ phức tạp:

- Trường hợp tốt nhất: $O(n^2)$
- Trường hợp xấu nhất: $O(n^2)$

Selection sort không ổn định (not stable) vì việc đổi chỗ có thể làm thay đổi thứ tự tương đối của các phần tử bằng nhau.

Ưu điểm của thuật toán là số lần hoán đổi ít (tối đa $n-1$), phù hợp khi chi phí hoán đổi lớn.

5.3 Insertion sort (Sắp xếp chèn)

Insertion sort mô phỏng cách con người sắp xếp bài trên tay. Ta lấy từng phần tử và chèn vào đúng vị trí trong phần đã sắp xếp.

Thuật toán hoạt động như sau:

- Xem phần tử đầu tiên là đã sắp xếp
- Lấy từng phần tử tiếp theo và chèn vào đúng vị trí trong phần đã sắp xếp

Ví dụ:

int a[] = {5, 2, 8, 1, 9};

Bước 1:

- Lấy 2, chèn vào trước 5 → {2, 5, 8, 1, 9}

Bước 2:

- Lấy 8 → giữ nguyên

Bước 3:

- Lấy 1 → chèn vào đầu → {1, 2, 5, 8, 9}

Cài đặt:

```
void insertionSort(int a[], int n) {
   for (int i = 1; i < n; i++) {
      int key = a[i];
      int j = i - 1;

      while (j >= 0 && a[j] > key) {
         a[j + 1] = a[j];
         j--;
      }

      a[j + 1] = key;
   }
}
```

Trong đó:

- key là phần tử cần chèn
- Các phần tử lớn hơn key sẽ được dịch sang phải

Độ phức tạp:

- Trường hợp xấu nhất: $O(n^2)$
- Trường hợp tốt nhất (mảng đã gần sắp xếp): $O(n)$

Insertion sort là thuật toán ổn định (stable).

Ưu điểm lớn nhất của insertion sort là hoạt động rất hiệu quả với dữ liệu gần như đã được sắp xếp. Vì vậy, trong thực tế, nó thường được sử dụng cho các mảng nhỏ hoặc làm bước cuối trong các thuật toán sắp xếp phức tạp hơn.